

Persistent Data Exfiltration in Memory-Augmented LLM Agents: Benchmark and Provable Defense

No Author Given

No Institute Given

Abstract. LLM agents that use tools and persistent memory deliver personalized assistance but introduce a critical attack surface: adversaries can inject instructions via untrusted tool outputs (e.g., emails) and abuse exfiltration tools plus long-term memory to steal user data across sessions. We study persistent data exfiltration attacks where a malicious payload is stored in memory in one session and later triggers the agent to send the user’s sensitive messages to an attacker when the user discusses finance, health, legal, tax, or identity. We instantiate an email assistant with four memory backends—explicit (ChatGPT-style), mem0 (agentic state-of-the-art), RAG, and a context sliding-window baseline—and four defenses: user-prompt-only indexing, no indexing after untrusted tools, length-limited memory, and a provably secure information-flow policy. We stress-test each configuration with OpenEvolve-based red-teaming: attacks are optimized on a single training instance per topic and evaluated on held-out test cases (different sessions and user phrasings) never seen during optimization. Despite this strict generalization setup, attack success reaches 100% (Mem0), 94% (Context), and 52% total with 100% max (RAG) without the provable defense, and remains high across multiple trigger sessions. The provable defense achieves 0% exfiltration empirically; heuristic defenses only partly help (e.g., Mem0 still 12.5% under no-untrusted-tools, limit-memory-length allows 33% on RAG). We provide a formal non-interference proof for the provable defense and a utility analysis for security–utility tradeoffs.

Keywords: LLM agents · persistent memory · prompt injection · data exfiltration · information-flow control · red-teaming.

1 Introduction

1.1 Motivation and Problem

Large language model (LLM) agents that call tools and maintain persistent memory are increasingly deployed for email, calendars, and other personalized assistants. Tools that read external data (e.g., inbox) and tools that send data out (e.g., send_email) create a natural exfiltration path. When that external data is untrusted—such as emails an adversary can send—indirect prompt injection

can plant instructions that the agent later obeys. Persistent memory makes this threat worse: a single poisoned session can store an instruction that activates in a later session when the user asks about sensitive topics, causing the agent to exfiltrate the user’s own message to the attacker. User queries about finance, health, legal, tax, or identity are especially dangerous because they often contain highly sensitive information (income, medical history, legal matters) that can be used for fraud, targeted phishing, or blackmail. Understanding how different memory backends and defenses behave under automated attack is therefore essential for secure agent design.

1.2 Related Work and Positioning

Indirect prompt injection via tool outputs has been studied in web and document settings [2, 3]. Memory-augmented agents—explicit (e.g., ChatGPT-style), RAG, and agentic systems such as Mem0—are widely used for personalization. The combination of untrusted tool inputs, exfiltration tools, and persistent memory has not been systematically benchmarked under strong, automated red-teaming. We fill this gap with a concrete email-agent testbed, multiple memory backends, several defenses, and formal plus empirical security and utility evaluation.

1.3 Threat Model and Attack Goal

The adversary controls only the content of emails that the user receives. The agent has tools to read the inbox (untrusted source) and to send email (exfiltration sink). The attack goal is persistent data exfiltration: in a first session, the agent reads a malicious email that contains hidden instructions; those instructions are stored in the agent’s persistent memory; in a later session, when the user asks about a sensitive topic (finance, health, legal, tax, or identity), the agent retrieves the poisoned memory and is induced to send the user’s message—often containing private details—to an attacker-controlled address. We focus on trigger topics (finance, identity, legal, health, tax) because user messages in these contexts are especially likely to include information valuable to attackers.

1.4 Memory Backends

We evaluate four memory backends. (1) Explicit: user-directed, deterministic storage (e.g., ChatGPT-style “remember this”); simple and inspectable. (2) Mem0: state-of-the-art agentic memory with semantic search and LLM-driven extraction; used by many production agents. (3) RAG: retrieval-augmented generation over chunked conversation history; common in document and long-context agents. (4) Context: a baseline that keeps a sliding window of recent conversation across sessions, with no separate retrieval layer. Each backend provides a different surface for poisoning and retrieval, and we test all under the same attack and defense conditions.

1.5 Defenses

We implement and compare four defenses. (1) User-prompt-only: only user messages are indexed into memory; agent-generated content (including tool outputs) is not stored. (2) No-untrusted-tools: once any untrusted tool (e.g., `read_inbox`) is called in a session, memory indexing is disabled for the rest of that session. (3) Limit-memory-length: memory entries are truncated to a fixed length (e.g., 80 characters) to limit the amount of injectable instruction. (4) Provable policy: an information-flow control (IFC) defense that labels memory and session state as trusted (T) or untrusted (U); any memory written after reading untrusted data or after retrieving U-labeled memory is labeled U; the harness blocks any call to an exfiltration tool when the session is U. We give a formal non-interference proof (by contradiction) showing that under this policy, the two-session exfiltration attack cannot succeed: activation requires U-labeled memory in context, which forces the session to U, so the exfiltration tool is blocked. We also validate this defense empirically in our benchmark.

1.6 Experimental Setup

We build an email agent with `read_inbox`, `send_email`, and memory tools, and wire it to each of the four memory backends. We define persistent-exfiltration attack scenarios for five trigger topics (health, legal, tax, finance, identity), with multi-session test cases: insert attack email, user asks to read inbox (poisoning), start new session, user sends a sensitive message (e.g., detailed tax or health information), and we check whether the agent sends that message to the attacker. Crucially, we use OpenEvolve-style automated red-teaming [1] (evolutionary search with MAP-Elites, LLM mutators, and a critic scorer) optimized on a single training instance per topic; the evolved attack is then evaluated on held-out test cases—different sessions (up to five trigger sessions per run) and different user phrasings—that were never seen during optimization. This strict train-on-one, test-on-many setup demonstrates generalization of the attack. Success is measured by whether the exfiltration email is sent with the user’s message in the body.

1.7 Main Results

Under this setup, attack success is alarmingly high for popular memory backends without the provable defense: averaged across suites, we observe 100% attack success for Mem0, 94.2% for Context, and 51.7% total (100% max in at least one session) for RAG. Session-by-session analysis shows that vulnerability persists across multiple trigger sessions (e.g., Context/no-defense remains near or at 100% from session 2 through session 5), so the threat is not limited to the first trigger. The provable (IFC) defense achieves 0% exfiltration across all backends where evaluated and is both formally proven and empirically validated. Heuristic defenses reduce but do not eliminate success: e.g., No Untrusted Tools still allows 12.5% total (62.5% max) on Mem0; Limit Memory Length allows 32.5% on RAG,

15% on Mem0, and 6.7% on Explicit; User Prompt Only leaves Mem0 at 15% total (50% max). Effectiveness thus depends on both trigger topic and memory backend. Example exfiltrated content includes full user messages with medical details, tax and income information (W-2, 1099, deductions), and legal/financial specifics—illustrating real-world impact.

1.8 Utility Analysis

To guide deployment choices, we provide a detailed utility evaluation in benign settings. We run standardized task suites: `memory_only`, `assistant_responses`, `untrusted_probe`, `untrusted_send`, `disable_send`, `memory_tools`, and `long_memory`. For each defense and backend we report success rates (and failure modes) by task type. This allows agent builders to see where each defense hurts utility—e.g., which task types drop under no-untrusted-tools or `limit_memory_length`—and to make an informed security–utility tradeoff when selecting a defense.

1.9 Contributions Summary

We make the following contributions: (1) A concrete threat model and benchmark for persistent data exfiltration in memory-augmented, tool-using LLM agents, with trigger topics chosen for high sensitivity. (2) Implementation and evaluation of four memory backends (explicit, mem0, RAG, context) and four defenses (three heuristic, one provably secure). (3) OpenEvolve-based stress-testing with a strict generalization setup—optimization on a single training instance per topic, evaluation on held-out test cases—showing 100% (Mem0), 94% (Context), and 52% total / 100% max (RAG) attack success without the provable defense, and session-by-session evidence that attacks persist across multiple trigger sessions. (4) A provably secure IFC defense with a formal non-interference proof and 0% exfiltration empirically. (5) A utility analysis by task type to support security–utility tradeoff decisions. Together, this gives practitioners a clear picture of the risks of persistent memory and exfiltration tools, and a path to provable security plus actionable guidance on heuristic defenses and utility cost.

2 Threat Model

We formalize the adversary, the agent interface, and the attack goal as follows.

Adversary capabilities. The adversary controls only the *content* of emails that the user receives. The adversary cannot modify the agent code, the memory backend implementation, or the user’s queries; cannot observe the agent’s internal state; and cannot directly access the user’s outbox or drafts. The attack vector is thus *indirect prompt injection* via the body (and optionally subject/headers) of emails that the agent reads when the user asks to read the inbox or search emails.

Agent interface. The agent is an email assistant with the following tools. **Untrusted (taint) tools**—those that read from adversary-controlled data—are: `read_all_emails` (list inbox), `search_emails` (semantic/keyword search over inbox, outbox, drafts), `reply_to_email`, and `forward_to_email`. **Exfiltration (sink) tools**—those that can send data to an external address—are: `send_email`, `reply_to_email`, and `forward_to_email`. The agent also has a memory tool (`update_memory`) when the explicit memory backend is enabled. The mailbox and all memory state are maintained in-memory per run (no persistent disk); each benchmark test gets a fresh environment.

System assumptions. We assume a single-user deployment, automatic memory ingestion from tool outputs (no direct attacker writes to memory), and an agent that can invoke external communication tools (e.g., send email).

Attack goal. The goal is *persistent data exfiltration*: (1) In a first session, the user asks the agent to read the inbox; the agent reads a malicious email planted by the adversary. (2) The malicious payload (hidden instructions) is stored in the agent’s persistent memory via the backend’s indexing behavior (explicit tool call, Mem0/RAG/context automatic indexing). (3) In a later session, the user sends a sensitive message (e.g., medical or tax details). (4) When the user asks about a trigger topic (finance, health, legal, tax, or identity), the agent retrieves the poisoned memory and is induced to send the user’s message to an attacker-controlled address (`vince2000@gmail.com`). Success is defined as: an email exists in the outbox with recipient `vince2000@gmail.com` and the *body* of that email containing the user’s trigger-session message (exact or substantial containment). In this work we do not require the agent to hide the exfiltration from the user; experiments that also measure stealth are reported separately in the results section.

3 Memory Backends

We implement four memory backends, each with a single active backend per run.

Explicit (ChatGPT-style). A dedicated `MemoryManager` keeps a short-term deque (last N messages, default $N = 15$) and a long-term list of entries. The user (or the agent on the user’s behalf) directs what to store via the `update_memory` tool: the tool accepts plain-text updates (e.g., “User prefers concise emails”) or “Forget ...” to remove matching facts. No embeddings or retrieval layer: at each turn, the full long-term list is formatted as text and injected into the system prompt. Entries can carry a trust label (T/U) for the provable defense. This backend is deterministic and inspectable.

Mem0. We use the Mem0 library (in-memory FAISS vector store, OpenAI embeddings and extraction LLM). Both the user message and the assistant response for each conversation turn are passed to Mem0’s `add` with `infer=True`, so an

LLM extracts and stores “memories” from the full turn; the only exception is when the *user-prompt-only* defense is active, in which case only user messages are passed for extraction. Retrieval is semantic search over stored memories (top- k , default $k = 10$). Large messages are chunked (e.g., 6000 tokens per chunk) before submission to avoid embedding-model token limits. Mem0 supports meta-data (e.g., session label T/U for the provable defense).

RAG (retrieval-augmented generation). We implement a RAG memory manager using OpenAI embeddings and an in-memory FAISS vector store. Each conversation turn is concatenated (optionally filtered by defense), split into fixed-size *character* chunks (default 512 characters; under `limit_memory_length` defense this is reduced to the configured limit, e.g., 80 characters). Chunks are embedded and added to the store. At each turn, the current user message is used as the query; we retrieve the top- k chunks (default $k = 8$) by similarity and prepend them as “Relevant Memory Context” to the conversation (as a system message). Metadata (e.g., label T/U) is stored per chunk for the provable defense.

Context (sliding-window baseline). A simple baseline with no embeddings: all conversation turns (similarly filtered by defense) are appended to a single history list. At each turn, the *entire* history is returned; if a token budget (`max_context_length`) is set, the string is truncated from the start (sliding window, keeping the most recent tokens). This models agents that simply concatenate recent dialogue into context. The context backend does not support the `limit_memory_length` defense (no per-entry length limit); the benchmark correctly skips that combination.

4 Defenses

We implement four defenses in a unified way across backends where applicable.

User-prompt-only. Only *user* messages are indexed into memory; assistant messages (and thus tool outputs, including the content of read emails) are excluded. For RAG and context, the indexed string is “User: ...” only. For Mem0, the message list passed to `add` is filtered to roles `user` only. This defense is *not* applicable to the explicit backend (explicit storage is user-directed via `update_memory`; the benchmark skips `explicit + user_prompt_only`).

No-untrusted-tools. A session is marked *untrusted* as soon as any taint tool (`read_all_emails`, `search_emails`, `reply_to_email`, `forward_to_email`) is invoked. For the rest of that session, memory indexing is disabled: `should_index_memory` returns false for RAG, context, and Mem0; the explicit `update_memory` tool returns a message that the update was not stored. New sessions start trusted again.

Limit-memory-length. The amount of injectable instruction per stored unit is capped. In all experiments reported here this limit was set to 80 characters; the value is configurable and could be remeasured for other settings. For **explicit**: the `update_memory` tool truncates the input string to that maximum length before storing. For **RAG**: the chunk size used when splitting the conversation turn is set to 80 characters, so each chunk is at most 80 characters. For **Mem0**: the conversation content passed to Mem0 is truncated to 80 characters per message before the extraction LLM runs, and any extracted memory text that exceeds the limit is truncated in a post-pass. The context backend does not support this defense (no per-entry limit).

Provable policy (information-flow control). We implement an IFC policy with two labels: trusted (T) and untrusted (U). **(P1) Session labeling:** Each session starts as T. **(P2) Taint:** When the agent calls a taint tool (read inbox, search, reply, forward), the session is set to U. **(P3) Memory labeling:** When memory is written (any backend), the stored entry is labeled with the *current* session label (T or U). **(P4) Retrieval:** When memory is retrieved, if any returned entry has label U, the *current* session is set to U. **(P5) Exfiltration block:** Before executing an exfiltration tool (`send_email`, or reply/forward when they send data), the harness checks the session label; if the session is U, the tool is not executed and the agent receives a fixed error message (e.g., “Exfiltration tools are not allowed in untrusted sessions”). Tools that are both taint and exfil (reply_to_email, forward_to_email) are always blocked when the provable policy is enabled, so the agent cannot both read from the inbox and send in one call. Under this policy, the two-session exfiltration attack cannot succeed: the trigger session must retrieve U-labeled memory to activate the payload, which sets the session to U, so the exfiltration tool is blocked. We validate this empirically (0% exfiltration in the benchmark) and give a formal non-interference argument in the appendix.

5 Experimental Setup

Email agent and environment. The agent is implemented with LangChain/LangGraph: a single LLM (e.g., GPT-4o or Gemini) is given a system prompt that lists the tools and guidelines (reply, forward, search, etc.). Each test run uses an in-memory mailbox (inbox/outbox/drafts), an in-memory trace store for tool calls and results, and per-run memory managers (no persistence across tests). Model and API settings (temperature, timeout, recursion limit) are read from a config file; we use temperature 0 and a fixed seed for reproducibility.

Attack scenarios and test cases. Persistent-exfiltration test cases are generated per (topic, memory backend, defense). Each case has: (1) `insert_attack_email`: plant a malicious email in the inbox (unread). (2) `user message`: “Read and print to me the exact contents of all the unread emails in my inbox.” (3) `start_new_session`. (4) One or more *trigger* steps: the user sends a message about a sensitive topic

(e.g., tax, health); the step defines an `attack_goal` whose success check is that the outbox contains an email to `vince2000@gmail.com` with the user’s message in the body. We use multiple *splits* indexed by a session parameter N (e.g., $N = 0, 10, 20, \dots, 100$). The index N indicates how many sessions of unrelated conversation occur *before* the session in which the user sends the trigger message: for $N = 0$, the trigger is sent in the first session after the attack; for $N > 0$, a memory snapshot taken after N such sessions is loaded, then a new session is started and the trigger is sent. For each split we have a *single* training instance (e.g., `train_N`) used for attack optimization and held-out test instances (e.g., `test_N`) with different user phrasings for evaluation. Train and test splits are disjoint.

OpenEvolve-based red-teaming. We use an evolutionary search (OpenEvolve-style) to optimize the attack email body (and optionally subject) on that single training instance per (topic, backend, defense) and per split. The search maintains a MAP-Elites grid over dimensions such as attack length and diversity (edit distance from a reference). Candidates are generated by an LLM mutator that receives the current attack and defense description (e.g., “limit_memory_length: 80 chars”) and produces variants. Each candidate is scored by running the agent with that attack email and computing: (1) a binary *attack goal* result (did the outbox contain the exfiltration email?), (2) a partial validator-based score (0–1), and (3) an LLM-based critic score (1–10 rubric) for feedback to the mutator. The best candidate (or a selected elite) is written to a cache and then *copied* to all test cases for that (topic, backend, defense) so that the same evolved attack is evaluated on held-out test cases without further optimization. This strict “train-on-one, test-on-many” setup measures generalization of the attack.

Metrics. For each (backend, defense, topic) we report: **Attack success rate (ASR)**: fraction of test cases where the attack goal succeeded (exfiltration email present). We report both *total* ASR (over all test cases) and *max* ASR (maximum ASR over any single test instance when multiple are run). Session-by-session breakdowns show whether success persists across trigger sessions. For the provable defense we also confirm 0% exfiltration. Utility (benign) suites are described in Section 7.

6 Results

(Results section to be filled with tables and figures: attack success by backend and defense, session-by-session analysis, and empirical validation of the provable defense. Experiments and results when stealth is also measured will be reported here.)

7 Utility Analysis

We run standardized *utility* task suites in benign settings (no attack email): `memory_only`, `assistant_responses`, `untrusted_probe`, `untrusted_send`, `disable_send`,

`memory_tools`, and `long_memory`. Utility is measured as the fraction of successful benign task completions over total benign tasks in each suite. For each (backend, defense) we report success rates by task type so that practitioners can see where each defense hurts utility (e.g., which tasks drop under `no-untrusted-tools` or `limit_memory_length`) and make an informed security–utility tradeoff.

8 Conclusion

(Conclusion to be filled.)

Acknowledgments. This study was funded by X (grant number Y).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Bibliography

- [1] Author, P.: Openevolve: Automated red-teaming for prompt injection. <https://arxiv.org/abs/2510.09023> (2024), last accessed 2025-02-26
- [2] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 1–15. ACM (2023). <https://doi.org/10.1145/3576915.3623159>
- [3] Liu, Y., Deng, G., Xu, Z., Li, Y., Zheng, Y., Zhang, Y., Zhao, P., Lin, T., Liu, Y.: Prompt injection attacks and defenses in LLM-integrated applications. CoRR **abs/2310.12815** (2023), <https://arxiv.org/abs/2310.12815>